ZKPriv: Zero-Knowledge Proofs That LLM Responses Contain No Prohibited Personal Data

Huzaif Khan*, Ghazal Mahdian†, Ali Jalooli‡
Department of Computer Science, California State University, Dominguez Hills, Carson, CA 90747, USA
*hkhan17@toromail.csudh.edu, †gmahdian1@toromail.csudh.edu, ‡ajalooli@csudh.edu

***Abstract*—Cloud large language models (LLMs) can reproduce personal data, and a provider that filters its outputs offers users and auditors no evidence that it did. We present ZKPRIV, which attaches to every LLM response a zero-knowledge proof that the response contains no personally identifiable information (PII) forbidden by the provider's registered policy, without revealing the prompt, the response or the model. Detection runs *inside* the circuit: 13 detectors, including a left-to-right Luhn automaton for payment cards, are compiled to minimized search automata, and the proof shows that none reaches an accepting state anywhere in the committed response. Names and places are handled by a second tier, in which an enclave signs named-entity labels that the circuit verifies. Implemented in Noir with zero-knowledge UltraHonk, a 256-byte response is proved in 0.87 s on a laptop CPU with the 12-detector strict circuit and verified in 12 ms. On held-out data, ZKPRIV beats Presidio, Presidio's pattern recognizers, spaCy NER and an LLM detector on structured PII (recall 0.955 vs. 0.855 for Presidio) while over-blocking far less than full Presidio (33.6% vs. 81.8%), and it refused 178 of 178 leaking responses from three LLMs.**

***Index Terms*—Large language models, zero-knowledge proofs, privacy compliance, PII detection, finite automata, verifiable computation**

I. INTRODUCTION

Cloud LLMs are increasingly used in healthcare, finance and customer service, where responses may only be released if they do not disclose regulated personal data. LLMs reproduce PII from their prompts, retrieved documents and training data [1]. A provider can filter outputs, but a filter offers no evidence to anyone outside the provider: a provider that disables it for one user, or for one request, looks the same to an auditor as one that never did.

Zero-knowledge (ZK) systems for LLMs such as zkLLM [2] and zkGPT [3] prove that a committed output came from a committed model, not whether it contains a Social Security number; guardrails [4], [5] check content but are not verifiable. Proving compliance over labels that the provider computes outside the proof does not help either: a dishonest prover sets every label to "not sensitive". ZKPRIV *computes* detection inside the circuit for structured PII, and requires names and places to be labeled and signed by an attested enclave over the same commitment.

ZKPRIV complements input-side protection such as our edge framework SenseCLLM [6], which anonymizes prompts before they reach the cloud: SenseCLLM controls what the model sees, ZKPRIV certifies what it returns. Zero knowledge matters not for the user, who sees the response, but for everyone else: a forwarded receipt tells an auditor that a response was compliant without revealing it, the prompt or the model, and the provider cannot later deny having certified it.

Contributions.

1) **PII detectors as absence proofs:** detectors compiled to minimized search automata, so "no match anywhere" is one check on the final state, including a left-to-right Luhn automaton for payment cards.
2) **A policy-bound receipt:** a Poseidon2 [7] commitment to the response, its length and padding, the request and a versioned policy, checkable by the user and verifiable by anyone.
3) **A second tier for semantic PII:** enclave-signed named-entity recognition (NER) labels verified in the circuit; tier 1 holds without trusting the enclave.
4) **A measured implementation** in Noir [8] with ZK UltraHonk [9], evaluated on held-out data against four baselines and on real LLM outputs, with all code and data released.[1]

II. RELATED WORK

Private and verifiable inference. Multi-party computation [10], [11], homomorphic encryption [12], [13], trusted execution environments (TEEs) [14] and prompt sanitization [15], [16] hide data from the provider [17]; zkLLM, zkGPT and FairProof [18] prove inference or model properties. None certifies an individual response's content; ZKPRIV's commitment can serve as the output commitment of such a proof.

ZK proofs over text. Reef [19] proves that a committed document matches, or does not match, a regular expression, and zk-regex [20] and Londhe *et al.* [21] prove regex matches for e-mail verification. Path-witness constructions prove that a match *exists*; they cannot prove absence. ZKPRIV applies a forced, whole-string automaton run, as in Reef's non-match case, to LLM responses, and adds what output compliance needs: detectors with checksum validation, length and padding binding, policy selection with versioning, and per-request binding. Zero-Knowledge Middleboxes [22] prove that encrypted traffic complies with a policy; ZKPRIV transfers that model to LLM responses.

Governance and filtering. Blockchain audit logs [23], [24] record that a check occurred, not that a response passed it; Presidio [25], hallucination checks [26] and SenseCLLM filter text without verifiable evidence.

[1] https://github.com/Huzzzaif/llm-pii-zkproof

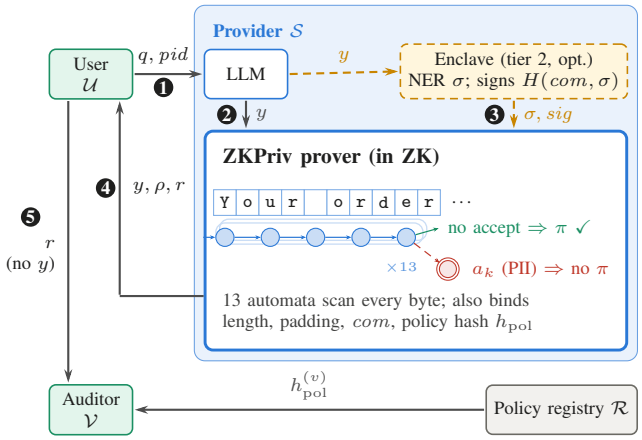


Fig. 1. Data flow. **1** The user sends prompt q and a fresh request id. **2** The provider's LLM answers y; the prover commits to it. **3** Optionally, an enclave runs named-entity recognition (NER) on y and signs the labels σ with the commitment. **4** The prover scans every byte with the automata of the enabled detectors; only if none reaches its accepting state does a proof π exist, and the provider returns y, the opening ρ and the receipt $r = (\pi, com, v, pid)$. **5** Any auditor verifies r without y, using the policy hash for version v from the public registry.

III. System Model

Principals. A user $\mathcal{U}$ sends prompt q with a fresh request identifier pid to a provider $\mathcal{S}$ that runs an LLM. $\mathcal{S}$ publishes policies in an append-only, hash-chained registry $\mathcal{R}$ of records $(v, h_{\text{pol}}^{(v)}, h_{\text{pol}}^{(v-1)}, \tau_v)$. An auditor $\mathcal{V}$ (a regulator or the user) holds the public verification key and reads $\mathcal{R}$ (Fig. 1).

Protocol. For response y (UTF-8 bytes, $|y| \le N$, the circuit's capacity), $\mathcal{S}$ samples a blinding value ρ and computes

$$\begin{aligned} com &= H\big(\text{pack}(y \,\|\, 0^{N-|y|}),\ |y|,\ \rho,\ pid\big), \\ h_{\text{pol}} &= H(m, v), \end{aligned} \tag{1}$$

where H is a Poseidon2 sponge over BN254 and m is the policy mask. $\mathcal{S}$ proves the statement of Section IV and returns (y, ρ, π, com, v). $\mathcal{U}$ recomputes com from the received y and ρ, checks that v is current, and verifies π with h_{pol} taken from the registry entry for v. A receipt (π, com, v, pid) can be forwarded to $\mathcal{V}$, who verifies it without y. ZKPriv makes compliance *checkable*, not enforced: a provider can always send a response without a proof, so clients treat an unreceipted response as non-compliant, much as a browser rejects a connection without a valid certificate.

Threat model. $\mathcal{S}$ is malicious: it may run any model, alter or skip filters, and try to certify a response other than the one delivered. The user performs its own checks honestly. We consider:

- *A1, false certification:* delivering y^* containing PII with a valid receipt. Prevented by soundness of the proof together with the user's commitment check, which requires a Poseidon2 collision to open com to a different string.
- *A2, policy downgrade:* proving against a weaker, older policy. Prevented by binding h_{pol} and v to the proof and checking v against the head of $\mathcal{R}$.
- *A3, replay:* reusing a receipt for another request. Prevented by binding pid into com.
- *A4, hiding past the end:* placing PII after the declared length. Prevented because the circuit forces every byte beyond $|y|$ to zero and binds $|y|$.
- *A5, label forgery (tier 2):* claiming that a response contains no names, or reusing labels from another response. Prevented because the labels must carry the enclave's signature over a digest that includes com.

Trust. Tier 1 (structured PII) needs no trust in a TEE or a classifier binary: the detectors are public circuit code. Tier 2 (names, places, other identifiers recognized by NER) trusts that the enclave's signing key is bound by remote attestation to the published NER code; a compromised enclave can under-label, but cannot weaken tier 1.

IV. Proving the Absence of Structured PII

A. From Detectors to Search Automata

Each detector D_k is a regular expression over bytes with explicit boundaries: an SSN must not be preceded or followed by an alphanumeric byte, so "1234-56-78901" is not reported. We write the start and end of the text as a virtual byte β. To prove that D_k occurs nowhere in y, we compile it to a deterministic finite automaton (DFA) for $\Sigma^* D_k$ by Thompson's NFA construction and Rabin–Scott subset construction [27], re-injecting the start state at every step so that matches beginning at any position, including overlapping ones, are tracked. We then make accepting states absorbing and minimize the DFA. After these steps

$$D_k \text{ occurs in } \beta y \beta \iff \delta_k^*(s_0, \beta y \beta) = a_k, \tag{2}$$

where a_k is the unique accepting state left by minimization. A witness that "skips" a match is impossible: every transition is fixed by the table, so the circuit only checks the final state.

ZKPriv ships 13 detectors: SSN; e-mail and obfuscated e-mail ("name AT host DOT com"); US and international (E.164-style) phone numbers; IPv4; numeric and written dates; dates of birth, i.e. a date within 20 characters of a cue such as "born" or "DOB"; payment cards; account numbers (8+ digits); prefixed identifiers (passport, record and account numbers such as `G34800996` or `MRN-315378`); and IBAN. A *standard* policy blocks birth dates only, a *strict* policy every date, following HIPAA Safe Harbor's treatment of dates.

To shrink the tables we partition the 256 byte values into classes with identical transition columns. A step costs two table lookups: byte to class, then (state, class) to the next state.

B. Payment Cards and the Luhn Check

Card numbers are 12–19 digits (ISO/IEC 7812; Maestro cards may have 12), optionally grouped by single spaces or hyphens, and valid numbers satisfy the Luhn checksum [28]. Luhn doubles every second digit *counted from the right*, which a left-to-right automaton cannot know in advance. We therefore track both possibilities. After digits $d_1 \dots d_k$, let S_0 be the

checksum if d_k is the last digit and S_1 the checksum if d_k is doubled. Reading digit d gives

$$S'_0 = d + S_1, \qquad S'_1 = \mathrm{dbl}(d) + S_0 \pmod{10}, \quad (3)$$

with $\mathrm{dbl}(d) = 2d - 9[d \geq 5]$. A run ending at a non-alphanumeric byte with $12 \leq k \leq 19$ and $S_0 \equiv 0$ is a card. The automaton has states $(k, S_0, S_1, \text{after-separator})$ plus idle, in-word and too-long states, which minimizes to 3,447 states.

C. The Circuit

The private inputs are the padded bytes $y_0 \dots y_{N-1}$, length $\ell = |y|$, blinding ρ, policy mask m and version v; pid is public, and (com, h_{pol}) are public outputs. The circuit enforces:

P1 *Length and padding:* $\ell \leq N$; $y_i \neq 0$ for $i < \ell$ and $y_i = 0$ for $i \geq \ell$.

P2 *ASCII guard (policy bit 13):* $y_i < 128$ for all i, which rejects Unicode digits and look-alike characters that byte-level detectors would otherwise miss.

P3 *Detectors (policy bits 0–12):* for each enabled D_k, $\delta_k^*(s_0, \beta y_{0..\ell-1}\beta) \neq a_k$; the transition at position i is applied only when $i < \ell$.

P4 *Commitment and policy:* com and h_{pol} are computed as in (1), packing 31 bytes per field element.

A response that violates the policy has no satisfying witness, so the prover fails before producing a proof.

D. Tier 2: Enclave-Signed Semantic Labels

Patterns cannot recognize names or places. In tier 2 an enclave attested to run published NER code (Presidio [25] with spaCy `en_core_web_sm`) labels y and sets a label mask σ with one bit each for PERSON, LOCATION, NRP (nationality, religious or political group) and other identifiers (passport, driver license, bank account). It computes $d = H(com, \sigma)$ and signs d with ECDSA over secp256k1, using a key bound to the enclave by remote attestation. The circuit additionally takes σ and the signature as private inputs and the enclave public key as a public input, and enforces

P5 *Signed labels:* $\mathrm{Verify}_{pk}(\mathrm{sig}, H(com, \sigma)) = 1$ and $\sigma \wedge m_{\mathrm{sem}} = 0$, where m_{sem} is the semantic part of the policy mask.

Because the digest contains com, labels signed for one response cannot be reused for another, and changing σ invalidates the signature. The enclave computes d with the same Poseidon2 code as the circuit, so the two cannot disagree.

Security. The relation is $\mathcal{L} = \{(pid, com, h_{\mathrm{pol}}) : \exists (y, \ell, \rho, m, v)$ satisfying P1–P4$\}$, with pk and P5 added in tier 2. By knowledge soundness of UltraHonk, a verifying proof yields such a witness. By (2) and the correctness of compilation, which we test differentially (Section V), the extracted y contains no match of any enabled detector. By collision resistance of Poseidon2, the user's opening (y, ρ) of com equals the extracted y, which gives A1; binding h_{pol}, v and pid gives A2 and A3, P1 gives A4, and unforgeability of ECDSA together with the commitment inside the signed digest gives A5. Zero knowledge of the UltraHonk ZK variant hides y and ρ from $\mathcal{V}$; the policy m is public through the registry.

TABLE I
R1: CORRECTNESS AND FORGERY CHECKS.

Check	Result
13 automata vs. independent `re`/Luhn reference on 300k random and pattern-seeded strings (64,747 matches)	3.9 M/3.9 M agree
Generated Noir circuit vs. reference (witness generation)	300/300 agree
Tier-2 forgeries: zeroed labels, flipped signature bit, other enclave's key, labels reused for another response	4/4 refused

The guarantee is relative to the detector language: it says that no *expressible* PII is present, not that no PII is present.

V. Evaluation

Setup. The detector compiler is written in Python and generates the Noir circuit (Noir 1.0.0-beta.22, Barretenberg 5.0.0-nightly.20260522, ZK UltraHonk with a Poseidon2 transcript). All timings use one MacBook Pro laptop (Apple M4 Pro, 24 GB), median of 10 runs, with the verification key computed once per circuit. We developed the detectors on 500 ai4privacy [29] records (train split) and 500 synthetic US-format records, then *froze* them and report results on held-out data fixed before any tuning: 500 records from the ai4privacy *validation* split and 500 synthetic records from a new seed. All systems use one protocol: a value is caught if any flagged span overlaps it, and a record is *over-blocked* if anything is still flagged after every annotated value is replaced by "X". Baselines are Presidio [25] with all recognizers at its default threshold, Presidio's pattern recognizers alone, spaCy NER, and an LLM detector (`llama3.2:3b` asked to list every PII substring, as in SenseCLLM [6]). We report four results, R1–R4.

A. R1: Correct and Unforgeable

In Table I, differential testing exposed two bugs during development, an e-mail boundary error and a mis-parsed `\n` escape in the regex compiler, both fixed before any reported measurement. The enclave is simulated: code and keys are real, hardware attestation is not reproduced.

B. R2: Cost

Policies compile to circuits, and the two we evaluate differ sharply in cost (Fig. 2); a 1 KiB response takes 1.31 s under the strict policy. The strict circuit, whose generic date detectors already block every date, needs no birth-date detector: with 12 detectors it has 221,744 gates at $N = 256$, proves in 0.87 s with 450 MiB and verifies in 12 ms, with detection identical to Fig. 3 on both held-out sets. The standard circuit adds the context-sensitive birth-date detector, whose "cue within 20 characters of a date" automaton accounts for 340,647 gates (61% of the circuit) and raises proving to 2.62 s (562,391 gates; 3.86 s for 1 KiB). Cost is dominated by committing these transition tables once, not by the per-byte scan, so it grows slowly with N. Tier 2 adds 37,986 gates for ECDSA

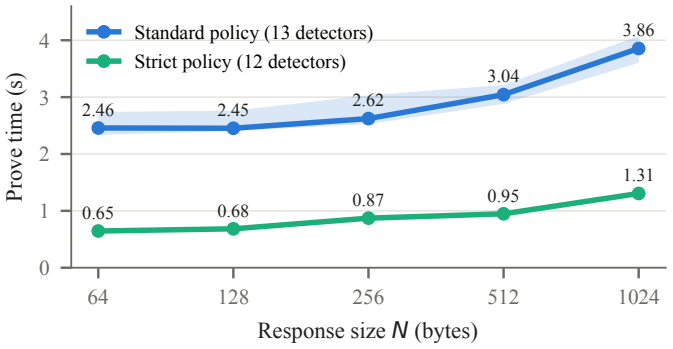


Fig. 2. R2: proving time against response size (median of 10 runs; shaded: interquartile range). Verification takes 9–12 ms (strict) and 19 ms (standard), and proofs are 14.3 KiB, at every size.

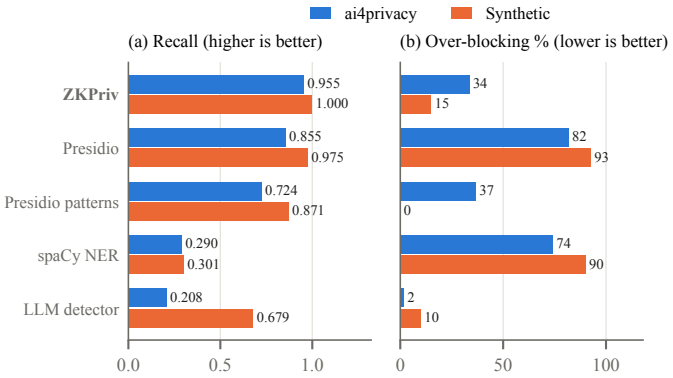


Fig. 3. R3: held-out detection of structured PII (SSN, e-mail, phone, card, date of birth, IP). (a) Recall. (b) Over-blocking: records still refused after every annotated PII value is replaced by "X". Other ID (Table II) is not included. ZKPRIV uses the strict policy.

verification (3.51 s on the standard circuit); NER and signing take 16 ms and 2.2 ms. The prover uses all cores, so parallel provers do not raise throughput (0.25 vs. 0.24 proofs/s with 1 and 8, including witness generation). Proving is shorter than generating the response: the median Llama 3.1 8B response took 4.7 s.

C. R3: Detection Against Four Baselines

On structured PII, ZKPRIV's in-circuit detectors beat every baseline in aggregate structured recall on both held-out sets (Fig. 3): strict recall is 0.955 (95% CI [0.92, 0.97]) on ai4privacy against 0.855 for Presidio, 0.724 for its pattern recognizers, 0.290 for spaCy and 0.208 for the LLM detector, and 1.000 on synthetic data against 0.975. It does so while over-blocking far less than Presidio (33.6% vs. 81.8%; 15.0% vs. 92.6%); Presidio's pattern recognizers over-block 36.8% and 0.0%. The strict policy's synthetic over-blocking comes from non-birth dates: the standard policy reaches 0.985 recall with 0.0% over-blocking on synthetic data. Over-blocking on ai4privacy comes mostly from identifiers the dataset leaves unannotated and, under the strict policy, non-birth dates. With all PII including names and places, tier 2 reaches 0.688 recall at 56.6% over-blocking against Presidio's 0.747 at 81.8%: tier 2 inherits the recall of the small NER model it runs, so names remain the weakest category.

TABLE II
R3 BY CATEGORY: HELD-OUT RECALL (TIER 1, STRICT POLICY; NAMES AND PLACES USE TIER 2). PAT.: PRESIDIO'S PATTERN RECOGNIZERS. BOLD: ZKPRIV STRICTLY BEST.

	ai4privacy				Synthetic			
Category	n	ZKPRIV	Presidio	Pat.	n	ZKPRIV	Presidio	Pat.
SSN / tax ID	47	0.83	0.98	0.96	325	1.00	1.00	1.00
E-mail	63	0.98	1.00	1.00	425	**1.00**	0.99	0.81
Phone	61	**0.98**	0.56	0.43	350	**1.00**	0.92	0.91
Card	20	**1.00**	0.90	0.80	50	**1.00**	0.90	0.90
Date of birth	30	**1.00**	0.93	0.33	225	1.00	1.00	0.66
IP address	–	–	–	–	125	1.00	1.00	1.00
Other ID	104	0.64	0.69	0.57	400	**1.00**	0.94	0.88
Name (tier 2)	222	0.54	0.70	–	475	0.93	0.99	–
Place (tier 2)	132	0.52	0.68	–	–	–	–	–

TABLE III
R4: REAL LLM RESPONSES (50 HELD-OUT RECORDS PER MODEL; 200 RESPONSES EACH).

Model	Leaking	Refused	General refused
Qwen 2.5 7B	144	144	0/50
Llama 3.1 8B	26	26	0/50
Llama 3.2 3B	8	8	0/50
Total	178	178	0/150

Table II breaks recall down by category. ZKPRIV's lead comes from international phone numbers with mixed dot, space and hyphen separators (e.g. `+44.88 299-2125`), which Presidio misses; from 18- and 19-digit cards; and from birth dates in month/year form (`October/17`). Presidio leads on ai4privacy's tax numbers, whose non-US formats (`77739.08202`) our US-centered SSN detector does not express, and on "Other ID", where our misses are ID cards with trailing letters (`IV45087AW`) and license numbers with embedded name fragments. Both gaps are detector-language gaps, not proof-system gaps: adding a pattern adds an automaton, and the proof covers it with no change to the protocol.

D. R4: Real LLM Outputs

Each model received 50 held-out synthetic records (temperature 0) and was asked to list the record's identifiers, to draft a reply confirming the contact details on file, or to repeat the record verbatim, plus one general question without a record. A response is *leaking* if it reproduces an identifier from the record. ZKPRIV refused every leaking response, catching 756 of the 756 reproduced values, and none of the 150 general answers (Table III). It also refused 7 responses with altered or invented identifiers. The Llama models mostly declined to list or repeat identifiers, while Qwen complied with almost every request.

E. Deployment

Changing a policy. A policy is a mask over the detectors compiled into a circuit, so changing it only appends a registry

record. Adding a detector, or specializing a circuit to a policy as our strict circuit does, yields a new verification key published with that record; the receipt's v thus identifies the detector set.

Adaptive evasion. A provider that wants to leak can re-encode a value. Unicode digits and look-alike characters fail the ASCII guard (P2), and "AT"/"DOT" e-mails are detected; digits spelled as words, values split across responses and base64 each need a further detector, at the cost of more over-blocking.

F. Limitations

ZKPRIV certifies the absence of PII that its two tiers express. Tier 2 is only as good as its NER model and trusts enclave attestation, which our prototype simulates. Our data are synthetic or semi-synthetic; responses longer than N need a larger circuit or chunked proofs; and per-response checks do not bound leakage across many queries. The guarantee also depends on the detector compiler, which we test differentially rather than verify formally.

VI. CONCLUSION

ZKPRIV turns an LLM provider's privacy promise into a receipt anyone can check: a zero-knowledge proof, computed in 0.87 s on a laptop CPU, that a response contains no PII forbidden by the registered policy, with detection inside the circuit rather than asserted by the provider. On held-out data it beats four baselines in structured-PII recall while over-blocking far less than full Presidio, and it refused every leaking response from three LLMs. Future work will run tier 2 on attested hardware, use stronger NER models, aggregate receipts across queries, and compose ZKPRIV with inference-correctness proofs.

REFERENCES

[1] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in *Proceedings of the 30th USENIX Security Symposium*. USENIX Association, 2021, pp. 2633–2650.

[2] H. Sun, J. Li, and H. Zhang, "zkLLM: Zero knowledge proofs for large language models," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. ACM, 2024, pp. 4405–4419.

[3] W. Qu, Y. Sun, X. Liu, T. Lu, Y. Guo, K. Chen, and J. Zhang, "zkGPT: An efficient non-interactive zero-knowledge proof framework for LLM inference," in *Proceedings of the 34th USENIX Security Symposium*. USENIX Association, 2025, pp. 2045–2063.

[4] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2023, pp. 431–445.

[5] H. Inan *et al.*, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," arXiv preprint arXiv:2312.06674, 2023.

[6] H. Khan, G. Mahdian, and A. Jalooli, "SenseCLLM: A three-agent edge-resident framework for adaptive PII detection and selective anonymization in IoT-to-LLM pipelines," in *IEEE World AI IoT Congress (AIIoT)*, 2026, pp. 1–7.

[7] L. Grassi, D. Khovratovich, and M. Schofnegger, "Poseidon2: A faster version of the Poseidon hash function," in *Progress in Cryptology – AFRICACRYPT 2023*, ser. LNCS, vol. 14064. Springer, 2023, pp. 177–203.

[8] Noir contributors, "Noir: A domain-specific language for zero-knowledge proofs," https://noir-lang.org, 2026, version 1.0.0-beta.22.

[9] Aztec Labs, "Barretenberg: UltraHonk proving system," https://github.com/AztecProtocol/aztec-packages/tree/next/barretenberg, 2026, version 5.0.0-nightly.20260522.

[10] Y. Dong, W.-j. Lu *et al.*, "PUMA: Secure inference of LLaMA-7B in five minutes," *Security and Safety*, vol. 4, p. 2025014, 2025.

[11] K. Gupta, N. Jawalkar, A. Mukherjee, N. Chandran, D. Gupta, A. Panwar, and R. Sharma, "SIGMA: Secure GPT inference with function secret sharing," *Proceedings on Privacy Enhancing Technologies*, vol. 2024, no. 4, pp. 61–79, 2024.

[12] L. De Castro, D. Escudero, A. Agrawal, A. Polychroniadou, and M. Veloso, "EncryptedLLM: Privacy-preserving large language model inference via GPU-accelerated fully homomorphic encryption," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025, pp. 12 677–12 688.

[13] D. Park, E. Lee, and J.-W. Lee, "Powerformer: Efficient and high-accuracy privacy-preserving language model with homomorphic encryption," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 11 090–11 111.

[14] M. Chrapek, M. Copik, E. Mettaz, and T. Hoefler, "Confidential LLM inference: Performance and cost across CPU and GPU TEEs," in *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, 2025, pp. 84–98.

[15] M. Tong *et al.*, "InferDPT: Privacy-preserving inference for closed-box large language models," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 5, pp. 4625–4640, 2025.

[16] Z. Zeng *et al.*, "PrivacyRestore: Privacy-preserving inference in large language models via privacy removal and restoration," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 10 821–10 855.

[17] D. Andreoletti, A. Rudi, E. Carpanzano, F. Lelli, and T. Leidi, "Privacy-preserving LLM inference in practice: A comparative survey of techniques, trade-offs, and deployability," Cryptology ePrint Archive, Paper 2026/105, 2026.

[18] C. Yadav, A. Roy Chowdhury, D. Boneh, and K. Chaudhuri, "FairProof: Confidential and certifiable fairness for neural networks," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024, pp. 55 682–55 705.

[19] S. Angel, E. Ioannidis, E. Margolin, S. Setty, and J. Woods, "Reef: Fast succinct non-interactive zero-knowledge regex proofs," in *Proceedings of the 33rd USENIX Security Symposium*, 2024, pp. 3801–3818.

[20] ZK Email, "zk-regex: Regex verification circuits," https://github.com/zkemail/zk-regex, 2025.

[21] S. Londhe, A. Gupta, S. Suegami *et al.*, "Zero-knowledge proofs of generalized regular expression matching for anonymized email verification," *Proceedings on Privacy Enhancing Technologies*, vol. 2026, no. 4, pp. 740–771, 2026.

[22] P. Grubbs, A. Arun, Y. Zhang, J. Bonneau, and M. Walfish, "Zero-knowledge middleboxes," in *Proceedings of the 31st USENIX Security Symposium*, 2022, pp. 4255–4272.

[23] S. Jan, H. A. Razzaqi, A. Akarma, and M. R. Belgaum, "A blockchain-monitored agentic AI architecture for trusted perception–reasoning–action pipelines," 2025, presented at the IEEE International Conference on Computing and Applications (ICCA 2025), Bahrain; arXiv:2512.20985.

[24] D. W. H. Ameen, "AI-driven secure audit trails for financial compliance using immutable blockchain logs," in *Proceedings of the 3rd International Conference on Cyber Resilience (ICCR 2025)*. IEEE, 2025, pp. 1–7.

[25] Microsoft Corporation, "Presidio: Data protection and PII anonymization SDK," GitHub, 2023, available: https://github.com/microsoft/presidio.

[26] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 9004–9017.

[27] M. O. Rabin and D. Scott, "Finite automata and their decision problems," *IBM Journal of Research and Development*, vol. 3, no. 2, pp. 114–125, 1959.

[28] H. P. Luhn, "Computer for verifying numbers," U.S. Patent 2,950,048, 1960.

[29] ai4privacy, "pii-masking-400k: Large-scale annotated PII masking dataset," Hugging Face Datasets, 2024, available: https://huggingface.co/datasets/ai4privacy/pii-masking-400k.